

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Mock Interview (Low)
Assessment Tool Weightage: 30%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Interviewer: 'Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?'	BL3 – Apply	5
2	Interviewer: 'What is the difference between heuristic optimization and cost-based optimization? When would you use each?'	BL4 – Analyze	5
3	Interviewer: 'How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.'	BL3 – Apply	5
4	Interviewer: 'What are ACID properties? Give a real-world example where each property is critical.'	BL3 – Apply	5
5	Interviewer: 'Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?'	BL4 – Analyze	5
6	Interviewer: 'What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.'	BL4 – Analyze	5
7	Interviewer: 'Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?'	BL4 – Analyze	5

8	Interviewer: 'Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?'	BL3 – Apply	5
9	Interviewer: 'What is a serializability schedule? How do you test for conflict serializability using a precedence graph?'	BL4 – Analyze	5
10	Interviewer: 'Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?'	BL4 – Analyze	5

Code of Conduct:

I K.MOHAMMED NAFEEZ [192571070] certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K.MOHAMMED NAFEEZ

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated

Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately

DETAILED ANSWERS – CO5 MOCK INTERVIEW

Q1. Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?

Answer:

Query processing is the complete process through which a DBMS converts an SQL query written by the user into an executable plan and produces the required result. The major stages are parsing, translation, optimization, plan selection, execution, and result generation.

1. Parsing and validation:

The DBMS first receives the SQL statement. The parser checks whether the SQL syntax is correct. It also checks table names, column names, data types, and other semantic details. If the query is invalid, an error is returned.

2. Query translation:

The valid SQL statement is converted into an internal representation, commonly based on relational algebra. Operations such as selection, projection, sorting, and joins are represented in a query tree.

3. Query decomposition:

The DBMS breaks the query into smaller operations. For example, WHERE conditions become selections and matching tables become join operations.

4. Query optimization:

The optimizer generates alternative execution plans. It can apply heuristic rules such as pushing selections and projections closer to the base tables, reducing the amount of data processed by later operations.

5. Cost estimation:

For cost-based optimization, the optimizer estimates the cost of each possible plan using statistics such as number of tuples, number of pages, available indexes, selectivity, memory, CPU requirements, and estimated disk I/O.

6. Choosing physical operators:

The optimizer decides whether to use an index scan or table scan and which join algorithm is suitable, such as nested-loop join, sort-merge join, or hash join.

7. Execution:

After selecting the best estimated plan, the execution engine runs the physical operators and obtains the required rows.

8. Result generation:

The final result is formatted and returned to the user.

Therefore, the query optimizer chooses a plan that is expected to provide the required result with low execution cost while considering available database resources and statistics.

Q2. What is the difference between heuristic optimization and cost-based optimization? When would you use each?

Answer:

Heuristic optimization and cost-based optimization are two approaches used by a DBMS to improve query performance.

Heuristic optimization:

Heuristic optimization uses predefined rules that generally improve query performance without calculating the exact cost of every possible plan.

Important heuristic rules include:

1. Push selection operations down toward base relations.
2. Perform projections early.
3. Reduce the number of tuples before performing joins.
4. Perform highly selective operations as early as possible.
5. Choose a join order that produces smaller intermediate relations.

For example, instead of joining two complete tables and then applying a WHERE condition, the DBMS can first filter the rows that satisfy the condition and then perform the join.

Advantages:

- Fast optimization.
- Simple to implement.
- Does not require detailed statistics.
- Useful for common query transformations.

Cost-based optimization:

Cost-based optimization generates multiple possible execution plans and estimates the cost of each one. It uses information such as table size, number of pages, indexes, selectivity, available memory, CPU cost, and disk I/O.

For example, the optimizer may compare:

- Nested-loop join.
- Hash join.
- Sort-merge join.
- Different join orders.
- Index scan versus full table scan.

It then selects the plan with the lowest estimated cost.

Difference:

Heuristic optimization → based mainly on general rules.

Cost-based optimization → based on estimated execution costs.

Heuristic optimization is useful when fast plan generation is important. Cost-based optimization is preferred when several significantly different plans are possible and reliable statistics are available. In practice, a DBMS can use heuristic transformations first and then cost-based optimization to select the physical execution plan.

Q3. How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.

Answer:

Database connectivity allows an application to communicate with a database for operations such as inserting, updating, deleting, and retrieving data. In a Java application, JDBC is normally used.

Implementation steps:

1. Install and configure the database server.
2. Create the required database and tables.
3. Add the appropriate JDBC driver to the Java project.
4. Specify the database URL, username, and password.
5. Create a database Connection.
6. Use PreparedStatement for parameterized SQL queries.
7. Execute the query.
8. Process the ResultSet for SELECT statements.
9. Commit the transaction when operations are successful.
10. Roll back when an exception occurs.
11. Close the ResultSet, statement, and connection, or return the connection to a connection pool.

Example:

```

Connection con = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/college",
    "root",
    "password"
);

PreparedStatement ps =
    con.prepareStatement(
        "SELECT sid, name, dept FROM student WHERE dept = ?"
    );

ps.setString(1, "CSE");
ResultSet rs = ps.executeQuery();

while (rs.next()) {
    System.out.println(
        rs.getInt("sid") + " " +
        rs.getString("name") + " " +
        rs.getString("dept")
    );
}

```

JDBC:

JDBC stands for Java Database Connectivity. It is an API used by Java applications to communicate with relational databases. It provides classes and interfaces such as Connection, PreparedStatement, Statement, and ResultSet.

ODBC:

ODBC stands for Open Database Connectivity. It is a standard database connectivity interface that allows applications to communicate with databases through ODBC drivers.

Main differences:

- JDBC is designed specifically for Java applications.
- ODBC is a language-independent database connectivity standard.
- JDBC uses Java APIs and JDBC drivers.
- ODBC applications use ODBC drivers.

- JDBC is generally preferred for Java projects because it integrates directly with Java.

For a Java-based real project, I would use JDBC with PreparedStatement and connection pooling, together with proper exception handling and transaction management.

Q4. What are ACID properties? Give a real-world example where each property is critical.

Answer:

ACID represents Atomicity, Consistency, Isolation, and Durability. These properties are fundamental to reliable transaction management in DBMS.

1. Atomicity:

Atomicity means that a transaction is treated as one complete unit. Either every required operation is completed or none of them should take effect.

Example:

Consider a bank transfer of ₹5,000 from Account A to Account B.

- ₹5,000 must be deducted from A.

- ₹5,000 must be added to B.

If the system deducts the amount from A but fails before adding it to B, the transaction must be rolled back. This ensures that the transfer is all-or-nothing.

2. Consistency:

Consistency means that a transaction must preserve all database rules, constraints, and relationships. The database should move from one valid state to another valid state.

Example:

In a banking system, account and transaction constraints must remain valid after a transfer. The transaction should not leave the database in a state that violates its integrity constraints.

3. Isolation:

Isolation means that concurrent transactions should not interfere in a way that exposes incorrect intermediate results.

Example:

Suppose T1 is transferring money between two accounts. Another transaction T2 checking balances should not observe an intermediate state where money has been deducted from one account but not yet added to the other.

4. Durability:

Durability means that once a transaction is successfully committed, its changes must survive system failures such as a crash or restart.

Example:

After a customer payment is committed, the payment record should still exist after the database server restarts.

Importance:

Atomicity protects transactions from partial execution.

Consistency protects database integrity.

Isolation controls problems caused by concurrent transactions.

Durability protects committed data from system failures.

Together, ACID properties make transaction processing reliable and predictable.

Q5. Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?

Answer:

Concurrency control is required because many transactions may execute at the same time. Without proper control, concurrent operations can produce incorrect or inconsistent results.

1. Dirty Read:

A dirty read occurs when transaction T2 reads data written by transaction T1 before T1 commits.

Example:

T1 changes account balance from ₹10,000 to ₹8,000 but has not committed.

T2 reads ₹8,000.

T1 then rolls back and the actual balance returns to ₹10,000.

T2 has therefore read data that was never committed.

2. Lost Update:

A lost update occurs when two transactions read the same value and both update it, causing one update to overwrite another.

Example:

The balance is ₹10,000.

T1 reads ₹10,000.

T2 also reads ₹10,000.

T1 changes it to ₹9,000.

T2 changes it to ₹12,000 based on its old copy.

The update made by T1 is lost.

3. Unrepeatable Read:

An unrepeatable read occurs when a transaction reads the same row twice and obtains different values because another transaction modified and committed the row between the two reads.

Example:

T1 reads an account balance of ₹10,000.

T2 updates it to ₹12,000 and commits.

T1 reads the same row again and gets ₹12,000 instead of ₹10,000.

Two-Phase Locking:

2PL divides transaction execution into two phases.

Growing phase:

- The transaction can acquire locks.
- It cannot release locks.

Shrinking phase:

- The transaction can release locks.
- It cannot acquire new locks.

Shared locks are used for reading and exclusive locks are used for writing. Conflicting operations cannot proceed simultaneously when the required locks are incompatible.

Strict 2PL is stronger because exclusive locks are retained until commit or abort. This prevents other transactions from reading or modifying data written by an uncommitted transaction and greatly simplifies recovery.

Therefore, 2PL provides serializable execution and controls common concurrency problems, while Strict 2PL additionally prevents cascading problems involving uncommitted writes.

Q6. What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.

Answer:

A deadlock occurs when a group of transactions wait for each other indefinitely because each transaction is holding a resource required by another transaction.

Example:

T1 holds a lock on A and requests B.

T2 holds a lock on B and requests A.

T1 waits for T2 and T2 waits for T1. Neither can proceed, so a deadlock exists.

Deadlock detection using Wait-For Graph:

1. Create a node for each active transaction.
2. If transaction T_i is waiting for a lock held by T_j , create an edge $T_i \rightarrow T_j$.
3. Continue adding edges for all waiting transactions.
4. Search the graph for a cycle.
5. If a cycle exists, the transactions involved in that cycle are deadlocked.
6. Select a victim transaction.
7. Roll back the victim and release its locks.
8. Allow the remaining transaction(s) to continue.

Example:

$T1 \rightarrow T2$

$T2 \rightarrow T3$

$T3 \rightarrow T1$

The cycle $T1 \rightarrow T2 \rightarrow T3 \rightarrow T1$ indicates a deadlock.

Prevention strategy 1 – Wait-Die:

Transactions are assigned timestamps.

- An older transaction requesting a lock held by a younger transaction may wait.
- A younger transaction requesting a lock held by an older transaction is rolled back.

Prevention strategy 2 – Wound-Wait:

- An older transaction requesting a lock held by a younger transaction can force the younger transaction to roll back.

- A younger transaction requesting a lock held by an older transaction waits.

Other methods include lock ordering and timeout-based approaches.

Deadlock detection is different from prevention: detection allows deadlocks to occur and identifies them, whereas prevention attempts to ensure that deadlocks cannot form.

Q7. Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?

Answer:

Recovery techniques restore the database to a consistent state after a system failure.

Immediate Update:

In Immediate Update, changes made by a transaction may be written to the database before the transaction commits. Therefore, the recovery system must be able to distinguish committed and uncommitted changes.

After a crash:

- Committed transactions may require REDO.
- Uncommitted transactions may require UNDO.

Example:

If T1 updates A from 100 to 150 and the new value reaches disk before T1 commits, then a crash can leave an uncommitted change on disk. Recovery must undo T1 if it did not commit.

Deferred Update:

In Deferred Update, changes are not applied to the database until the transaction commits. Therefore, uncommitted transaction changes are not present in the database pages.

After a crash:

- Committed transactions are REDO operations.
- Uncommitted transactions can be ignored.
- UNDO is not required for database changes that were never written.

Comparison:

Immediate Update:

- Changes can reach disk before commit.
- Uses UNDO and REDO.

- More flexible for page management.
- Requires careful write-ahead logging and recovery.

Deferred Update:

- Database is updated after commit.
- Uses NO-UNDO/REDO recovery.
- Simpler recovery because uncommitted changes do not need to be undone.
- May delay writing updates until commit.

Scenario preference:

Immediate Update is useful when the DBMS needs to write modified pages before transactions commit and wants flexibility in buffer management.

Deferred Update is useful when a simpler recovery model is desired and the system can defer applying database changes until commit.

The major difference is that Immediate Update may require both UNDO and REDO, whereas Deferred Update requires REDO for committed transactions and does not require UNDO for uncommitted database changes.

Q8. Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?

Answer:

Shadow Paging is a recovery method that maintains a stable copy of the page table and a working copy.

Two page tables are maintained:

1. Shadow Page Table:

It points to the pages belonging to the last committed and consistent database state. It is not changed during normal transaction processing.

2. Current Page Table:

It contains the page mappings used by the active transaction. When a page is modified, the original page is preserved and the modified information is written to a new page. The current page table is then changed to point to the new page.

Working:

Suppose the database initially has pages P1, P2, and P3.

- The shadow table points to these original pages.
- A transaction modifies P2.

- A new page P2' is created.
- The modified data is written to P2'.
- The current page table points to P2'.
- The shadow page table still points to P2.

Commit:

After successful completion, the current page table becomes the committed page table.

Failure:

If the system crashes before commit, the current page table is discarded and the shadow page table is used. The database therefore returns to the previous consistent state.

Advantages:

1. Recovery is conceptually simple.
2. Rollback can be performed by restoring the shadow page table.
3. Individual UNDO processing of changed pages is not necessary.
4. The original committed pages remain protected until commit.

Limitations:

1. New pages must be allocated for modified pages.
2. This can cause storage overhead and fragmentation.
3. Page-table management can become expensive for large databases.
4. It may be less suitable for high-concurrency systems.
5. Log-based systems provide more flexible recovery and are commonly used in large database systems.

Comparison:

Shadow paging mainly uses page-table switching for recovery, while log-based recovery records changes in a log and uses operations such as UNDO and REDO.

Shadow paging is simple, but log-based recovery is generally more flexible and scalable for large, highly concurrent database systems.

Q9. What is a serializability schedule? How do you test for conflict serializability using a precedence graph?

Answer:

A schedule is an ordering of operations from multiple transactions. A schedule is serial if one transaction completes before another begins. A serializable schedule is a concurrent schedule whose final effect is equivalent to some serial schedule.

Conflict serializability focuses on conflicting operations.

Two operations conflict when:

1. They belong to different transactions.
2. They operate on the same data item.
3. At least one operation is a write.

The main conflict pairs are:

- READ followed by WRITE.
- WRITE followed by READ.
- WRITE followed by WRITE.

Precedence graph construction:

1. Create one node for each transaction.
2. Scan the schedule from beginning to end.
3. For every pair of conflicting operations, if T_i 's operation occurs before T_j 's operation, add an edge $T_i \rightarrow T_j$.
4. Continue until all conflicts have been considered.
5. Check the graph for cycles.

Example:

Suppose:

T_1 : R(A), W(A)

T_2 : R(A), W(B)

T_3 : R(B), W(B)

Conflicts may produce:

$T_1 \rightarrow T_2$

$T_2 \rightarrow T_3$

The graph is:

$T_1 \rightarrow T_2 \rightarrow T_3$

There is no cycle, so the schedule is conflict-serializable. A possible serial order is T_1, T_2, T_3 .

If the graph were:

$T1 \rightarrow T2$

$T2 \rightarrow T3$

$T3 \rightarrow T1$

then a cycle would exist. The schedule would not be conflict-serializable.

Therefore:

No cycle $\rightarrow$ conflict-serializable.

Cycle $\rightarrow$ not conflict-serializable.

A topological ordering of an acyclic precedence graph provides an equivalent serial transaction order.

Q10. Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?

Answer:

Two-Phase Locking (2PL) is a concurrency-control protocol that uses locks to ensure conflict-serializable transaction schedules.

1. Basic 2PL:

Basic 2PL has two phases.

Growing phase:

- The transaction may acquire new locks.
- It cannot release locks.

Shrinking phase:

- The transaction may release locks.
- It cannot acquire new locks.

Once a transaction releases its first lock, it cannot obtain another lock.

Advantage:

Basic 2PL guarantees conflict serializability.

Limitation:

It can cause deadlocks because transactions may wait for locks held by each other. Also, locks can potentially be released before commit, which can lead to cascading rollback situations.

2. Strict 2PL:

Strict 2PL keeps all exclusive locks until the transaction commits or aborts. Therefore, other transactions cannot read or write data that has been modified by an uncommitted transaction.

Advantages:

- Prevents dirty reads of uncommitted writes.
- Prevents cascading rollback caused by reading uncommitted data.
- Simplifies transaction recovery.
- Guarantees conflict serializability.

3. Rigorous 2PL:

Rigorous 2PL is even stronger. Both shared locks and exclusive locks are held until the transaction commits or aborts.

Therefore:

Basic 2PL → locks can be released during the shrinking phase.

Strict 2PL → exclusive locks are held until commit/abort.

Rigorous 2PL → all shared and exclusive locks are held until commit/abort.

Which is commonly used?

Strict 2PL is commonly used because it provides a strong balance between correctness, recoverability, and concurrency. By retaining exclusive locks until commit or abort, it prevents other transactions from using uncommitted updates and makes recovery easier.

Rigorous 2PL provides even stronger locking but may reduce concurrency because shared locks are also held until the transaction completes.

Conclusion:

Basic 2PL provides serializability.

Strict 2PL provides serializability and strong recoverability properties.

Rigorous 2PL provides the strongest lock-retention rule among the three.

For practical database transaction processing, Strict 2PL is a widely useful choice because it provides strong correctness and recovery behavior without requiring every shared lock to be held until commit.

